

Tutorato Programmazione 1

20/10/25

1.

Scrivi una funzione ricorsiva che stampa i numeri interi da 0 a N, dato N in input.

2.

Scrivi un programma che prende in input un intero N e tramite **una** funzione ricorsiva stampa una sequenza di numeri da 0 a N, dove prima ci sono i numeri pari in ordine crescente e poi i numeri dispari in ordine decrescente.

Hint: fate attenzione all'ordine di quando stampare il numero o chiamare la funzione

Es:

N = 10

0 2 4 6 8 10 9 7 5 3 1

3.

Scrivere una funzione che, dato in input un intero N, per N volte ricorsivamente chieda all'utente di inserire un carattere.

Le lettere minuscole valgono 5 punti, quelle maiuscole 10 e tutti gli altri caratteri 1.

La funzione deve ritornare la somma dei punteggi, stamparla poi a video.

4.

Scrivere una funzione ricorsiva che, dati in input due numeri interi positivi, trovi il Massimo Comun Divisore di questi.

5.

Scrivere una funzione ricorsiva che calcoli il fattoriale di un numero intero positivo.

Riscrivere poi la sua versione iterativa, quale delle due è più efficiente?

Bonus:

Usare la libreria chrono (`#include <chrono>`) per misurare quanto dura l'esecuzione di ogni funzione.

Esempio:

```
chrono::time_point<chrono::steady_clock> start, end;  
chrono::duration<double> time_span;  
  
start = chrono::steady_clock::now();  
  
// chiamata alla funzione  
  
end = chrono::steady_clock::now();  
time_span = chrono::duration_cast<chrono::duration<double>>(end - start);  
cout << "Tempo trascorso: " << time_span.count() << " secondi" << endl;
```

Prendendo il tempo prima e dopo la funzione che si vuole misurare (start e end) è possibile sottrarli e ottenere la durata.

6.

Scrivi un programma che simuli un attacco n contro m a RisiKo!

1. Dare in input n e m, compresi tra 1 e 10.
2. Chiedi all'utente con quanti carri-armati vuole attaccare (max 3), mentre il difensore difende sempre con il massimo disponibile (max 3).
3. Confronta il dado più alto dell'attaccante con il più alto del difensore e così via.
4. Stampa il risultato dell'attacco.
5. Ripete l'attacco finchè o l'attaccante non può più attaccare (attaccante con un solo carroarmato) oppure il difensore è sconfitto (rimane con zero carroarmati).

Note implementative:

- Scrivere una funzione che simuli il singolo lancio di un dado.
- Scrivere una funzione che gestisca un turno di attacco

`attack(n_dadi_att, n_dadi_def, lost_attack, lost_def)`

(pensate bene a quali parametri passare per puntatore/riferimento e quali no)

- Per eseguire il matching tra i dadi dell'attaccante e del difensore è utile utilizzare una funzione di supporto `sort(a, b, c)` che ordini le variabili in ordine decrescente
- Controllate sempre che sia l'attaccante che il difensore non attacchino mai con più dadi che carroarmati!

Bonus:

Prova a implementare questo programma senza il limite di carri-armati con cui si può attaccare e usando un array per gestire i lanci dei dadi.